// SECURITY AUDIT REPORT

Monetrix

Smart Contract Security Assessment · v1.0

AUDITOR	DATE	PLATFORM	VERSION
Putfor	May 2026	Code4rena	v1.0

01 PROTOCOL SUMMARY

Monetrix is a USDC-backed synthetic dollar protocol deployed on HyperEVM. Users deposit USDC into `MonetrixVault.sol` and receive USDM 1:1. USDM holders can stake into `sUSDM.sol`, an ERC-4626 yield-bearing wrapper with cooldown-based unstaking and physical USDM isolation in `sUSDMEscrow.sol`.
USDM backing is tracked across EVM-side USDC balances and HyperCore-side positions, including spot balances, whitelisted hedge assets, supplied balances, perp account value, and HLP equity. `MonetrixAccountant.sol` reads this state through HyperCore precompiles and enforces settlement gates before yield can be declared. Yield is settled by the Operator, held in `YieldEscrow.sol`, and later distributed through `MonetrixVault.distributeYield()`: the user share is injected into sUSDM, while the remaining portions are routed to the Insurance Fund and Foundation. Authorization is centralized through `MonetrixAccessController.sol`, and the main protocol contracts are UUPS-upgradeable.

> *This audit report is provided for informational purposes only. I am an independent security researcher, and this engagement represents a best-effort review of the in-scope code. While I strive to identify as many vulnerabilities as possible, this report does not constitute a guarantee that the protocol is free of bugs, vulnerabilities, or other issues. The findings reflect the state of the code at the commit hash listed in Audit Details and may not account for changes introduced after the review period.*

02 RISK CLASSIFICATION

All findings are classified using the CodeHawks severity matrix.
Likelihood measures how probable exploitation is under realistic conditions.
Impact measures the potential damage to the protocol, its users, or its funds if the vulnerability is successfully exploited.

Likelihood / Impact	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

03 AUDIT DETAILS

Commit Hash: `3d94be1361ca01d959f9165a78f0d75c5657fe3e`

Scope

```
src/
├── core/
│   ├── ActionEncoder.sol # 131 nSLOC
│   ├── InsuranceFund.sol # 38 nSLOC
│   ├── MonetrixAccountant.sol # 220 nSLOC
│   ├── MonetrixConfig.sol # 162 nSLOC
│   ├── MonetrixVault.sol # 438 nSLOC
│   ├── PrecompileReader.sol # 129 nSLOC
│   ├── RedeemEscrow.sol # 53 nSLOC
│   ├── TokenMath.sol # 61 nSLOC
│   └── YieldEscrow.sol # 34 nSLOC
├── governance/
│   ├── IMonetrixAccessController.sol # 9 nSLOC
│   ├── MonetrixAccessController.sol # 37 nSLOC
│   └── MonetrixGovernedUpgradeable.sol # 36 nSLOC
├── interfaces/
│   ├── HyperCoreConstants.sol # 27 nSLOC
│   ├── IHyperCore.sol # 21 nSLOC
│   ├── IMonetrixAccountant.sol # 6 nSLOC
│   ├── IRedeemEscrow.sol # 9 nSLOC
│   └── IYieldEscrow.sol # 5 nSLOC
└── tokens/
    ├── USDM.sol # 48 nSLOC
```

```
├── sUSDM.sol # 237 nSLOC
└── sUSDMEscrow.sol # 25 nSLOC
```

Compatibilities

SOLIDITY	CHAINS
0.8.27	HyperEVM
TOKENS	**TIMELINE**
USDC, USDM, sUSDM	April 24 - May 4 2026

Roles

ROLE	DESCRIPTION
Default Admin	Grants / revokes all roles; authorizes ACL upgrade. Sits behind a 48h timelock.
Upgrader	Authorizes UUPS upgrade of all 9 proxies (Vault, USDM, sUSDM, Config, Accountant, RedeemEscrow, YieldEscrow, InsuranceFund, ACL). Sits behind a 48h timelock.
Governor	All Config / Accountant / Vault setters; `InsuranceFund.withdraw` Vault emergency paths (`emergencyRawAction` , `emergencyBridgePrincipalFromL1` — intentionally bypass both pause flags). Sits behind a 24h timelock.
Operator	Bridge, hedge, HLP, BLP, yield pipeline (`settle` / `distributeYield`), `fundRedemptions` , `reclaimFromRedeemEscrow` . Code-bounded: Operator can only move funds among Vault ↔ L1 own account / Vault ↔

	Escrows / sUSDM / InsuranceFund / Foundation. All destination addresses are pre-set by Governor. Operator cannot route funds to an external EOA or arbitrary address.
Guardian	Two independent pause switches: `pause` freezes user flows + mixed paths (`deposit` / `redeem` / `keeperBridge` / `settle` / `distributeYield`); `pauseOperator` freezes all Operator paths. No fund authority.
Vault (contract)	Via `onlyVault` : `USDM.mint/burn` , `sUSDM.injectYield` , Escrow fund movements, `Accountant.settleDailyPnL` / `notifyVaultSupply` .

Tools Used

Manual review · Foundry (testing & PoC) · Slither · Aderyn

04 EXECUTIVE SUMMARY

Methodology

The audit began with a full read-through of the core protocol contracts to understand the intended USDC -> USDM -> sUSDM lifecycle and the HyperCore-backed strategy model. The first pass mapped all user-facing flows: deposits, redemptions, sUSDM staking, cooldown-based exits, and yield distribution.

The second pass focused on cross-contract state synchronization. Particular attention was paid to two-phase user flows, such as `requestRedeem()` / `claimRedeem()` and `cooldownShares()` / `claimUnstake()`, where user state is created in one transaction and consumed later after a cooldown. This review also covered mutable dependency pointers such as `redeemEscrow` and `yieldEscrow`.

The third pass reviewed economic invariants around the Accountant's four-gate settlement pipeline, ERC-4626 share-price behavior, reward distribution timing, rounding direction, and HyperCore oracle/precompile dependencies. Operator and Governor powers were treated as trusted but high-blast-radius roles, with attention to cases where honest misconfiguration could still strand funds or desynchronize accounting.

Additional review was performed on pause semantics, proxy upgrade safety, storage layout assumptions, token compatibility assumptions, and edge-case staking behavior. Proof-of-concept tests were developed in the Code4rena template by Monetrix at `test/c4/C4Submission.t.sol` for the issues that were practical to demonstrate in Foundry.

Findings Overview

SEVERITY	COUNT
Medium	1
Low	2

05 FINDINGS

M-01 · MED

Pending redemptions can become unclaimable after `redeemEscrow` is changed

CONTRACT MonetrixVault.sol ROOT CAUSE Pending redemption state is not bound to the escrow that recorded the obligation

L-01 · LOW

Settled yield can become stranded after `yieldEscrow` is changed

CONTRACT MonetrixVault.sol ROOT CAUSE Settled yield is not bound to the escrow that received it

L-02 · LOW

Dust first staker can bypass the empty-sUSDM yield reroute and capture most user-side yield

CONTRACT MonetrixVault.sol ROOT CAUSE Empty-vault yield protection checks only `totalSupply() == 0`, not economically meaningful stake

M-01 · MEDIUM

Pending redemptions can become unclaimable after `redeemEscrow` is changed

SEVERITY	Medium
ROOT CAUSE	Pending redemption state is not bound to the escrow that recorded the obligation
CONTRACT	MonetrixVault.sol
FUNCTIONS	requestRedeem(uint256), claimRedeem(uint256), setRedeemEscrow(address)
LINES	L183-L211, L502-L505

Vulnerability Details

The redemption flow in `MonetrixVault` is split into two phases:

1. A user calls `requestRedeem(uint256)`, transfers USDM to the Vault, and creates a pending redemption request.
2. After `redeemCooldown`, the user calls `claimRedeem(uint256)` and receives USDC from `RedeemEscrow`.

When a redemption is requested, the obligation is recorded in the current `redeemEscrow`:

SOLIDITY MonetrixVault.sol

```solidity
function requestRedeem(uint256 usdmAmount) external nonReentrant whenNotPaused requireWired returns (uint256 requestId) {
    require(usdmAmount > 0, "zero amount");
    IERC20(address(usdm)).safeTransferFrom(msg.sender, address(this), usdmAmount);
@>  IRedeemEscrow(redeemEscrow).addObligation(usdmAmount);

    requestId = nextRedeemId++;
    redeemRequests[requestId] = RedeemRequest({
        owner: msg.sender,
        cooldownEnd: SafeCast.toUint64(block.timestamp + config.redeemCooldown()),
        usdmAmount: usdmAmount
    });
    _userRedeemIds[msg.sender].push(requestId);
    emit RedeemRequested(requestId, msg.sender, usdmAmount, block.timestamp + config.redeemCooldown());
}
```

However, the `RedeemRequest` struct does not store the address of the escrow that received the obligation:

SOLIDITY MonetrixVault.sol

```solidity
struct RedeemRequest {
    address owner;
    uint64  cooldownEnd;
    uint256 usdmAmount;
}
```

Later, when the user claims the redemption, the Vault again uses the current `redeemEscrow` address:

SOLIDITY MonetrixVault.sol

```solidity
function claimRedeem(uint256 requestId) external nonReentrant whenNotPaused requireWired {
    RedeemRequest memory req = redeemRequests[requestId];
    require(
        req.usdmAmount > 0
            && msg.sender == req.owner
            && block.timestamp >= req.cooldownEnd,
        "invalid claim"
    );
    uint256 amount = req.usdmAmount;
    delete redeemRequests[requestId];
    _removeUserRedeemId(req.owner, requestId);

    usdm.burn(amount);
@>  IRedeemEscrow(redeemEscrow).payOut(msg.sender, amount);
    emit RedeemClaimed(requestId, msg.sender, amount);
}
```

The `redeemEscrow` address is mutable by the Governor:

SOLIDITY MonetrixVault.sol

```solidity
function setRedeemEscrow(address _escrow) external onlyGovernor {
    require(_escrow != address(0), "zero address");
@>  redeemEscrow = _escrow;
    emit RedeemEscrowUpdated(_escrow);
}
```

As a result, if governance changes `redeemEscrow` between `requestRedeem()` and `claimRedeem()`, the claim no longer uses the escrow that originally recorded the user's obligation.
This causes pending redemptions to become unclaimable or, in some cases, to be paid from the wrong escrow.

Scenario

1. Alice deposits USDC into the protocol through `MonetrixVault.deposit()` and receives USDM.
2. Alice calls `requestRedeem(1_000e6)`.
3. The Vault transfers Alice's `1_000 USDM` to itself.
4. The Vault calls `oldRedeemEscrow.addObligation(1_000e6)`.
5. Alice's `RedeemRequest` is created, but it stores only `owner`, `cooldownEnd`, and `usdmAmount`. It does not store `oldRedeemEscrow`.
6. Before Alice's cooldown expires, Governor calls `setRedeemEscrow(newRedeemEscrow)`.
7. Alice waits until the cooldown expires and calls `claimRedeem(requestId)`.
8. The Vault calls `newRedeemEscrow.payOut(alice, 1_000e6)`.
9. Since Alice's obligation was recorded in `oldRedeemEscrow`, `newRedeemEscrow.totalOwed` does not include Alice's redemption.
10. `RedeemEscrow.payOut()` attempts to execute `totalOwed -= amount`, which reverts due to underflow if `newRedeemEscrow.totalOwed < amount`.
11. Alice's claim is stuck until governance manually restores the old escrow or performs a custom migration.

This is especially problematic because Alice has already transferred her USDM to the Vault during `requestRedeem()`. From Alice's point of view, the redemption flow was entered correctly, but the second phase becomes impossible due to a later configuration change.

Likelihood

Low. The vulnerability requires a privileged `GOVERNOR` action and is not a scenario that can be repeatedly or permissionlessly triggered by an arbitrary attacker. It is most likely to occur during an intended escrow migration or operational reconfiguration.

Likelihood is Low because the bug depends on an uncommon administrative action being performed while pending redemptions exist.

Impact

Medium. Users with pending redemptions can be unable to claim their USDC after the cooldown period. Their USDM remains locked in the Vault, while the corresponding obligation and USDC liquidity remain stranded in the old escrow.
In the best case, governance can recover by pointing the Vault back to the old escrow.
In the worse case, if a new escrow is already used by other users, claims may be paid from the wrong escrow, corrupting redemption accounting between old and new escrow instances.
Impact is Medium because user redemptions can be blocked and escrow accounting can become inconsistent, but recovery is possible through governance intervention.

LIKELIHOOD	IMPACT	OVERALL
Low	**Medium**	**Medium**
Dependent on the actions of the privileged role.	Some users' tokens may be frozen.	Medium × Low.

Proof of Concept

The PoC was added to the official Code4rena template:

SOLIDITY

```
test/c4/C4Submission.t.sol
```

The test performs the following steps:

SOLIDITY Proof of Code

```solidity
function test_submissionValidity() public {
    uint256 redeemAmount = 1_000e6;

    // Step 1: user deposits USDC and receives USDM 1:1.
    _deposit(user1, redeemAmount);
    assertEq(usdm.balanceOf(user1), redeemAmount, "user received USDM");

    // Seed the original escrow with USDC so a normal claim would be payable.
    // This isolates the failure to the escrow pointer switch, not liquidity.
    usdc.mint(address(redeemEscrow), redeemAmount);

```

```
    // Step 2: user opens a redeem request. The obligation is recorded in
    // the *current* RedeemEscrow, but the request stores only owner/time/amount.
    uint256 requestId = _requestRedeem(user1, redeemAmount);
    assertEq(usdm.balanceOf(user1), 0, "USDM is now locked in the vault");
    assertEq(redeemEscrow.totalOwed(), redeemAmount, "old escrow recorded the obligation");

    // Step 3: governance switches the vault to a fresh RedeemEscrow before
    // the user can claim. Pending requests are not bound to their original escrow.
    RedeemEscrow newRedeemEscrow = RedeemEscrow(
        address(
            new ERC1967Proxy(
                address(new RedeemEscrow()),
                abi.encodeCall(RedeemEscrow.initialize, (address(usdc), address(vault), address(acl)))
            )
        )
    );

    vm.prank(admin);
    vault.setRedeemEscrow(address(newRedeemEscrow));

    assertEq(address(vault.redeemEscrow()), address(newRedeemEscrow), "vault now points to new escrow");
    assertEq(newRedeemEscrow.totalOwed(), 0, "new escrow has no matching obligation");

    // Step 4: after the cooldown expires, the user tries to claim.
    // claimRedeem uses the *current* RedeemEscrow, so newEscrow.payOut()
    // underflows totalOwed and reverts. The user's redemption is stuck
    // until governance manually restores/migrates the old escrow.
    vm.warp(block.timestamp + config.redeemCooldown() + 1);

    vm.prank(user1);
    vm.expectRevert();
    vault.claimRedeem(requestId);

    // Final state proves the bug: the pending request still exists, the
    // user's USDM remains locked in Vault, and the old escrow still carries
    // the obligation/liquidity that claimRedeem no longer references.
    (address owner,, uint256 usdmAmount) = vault.redeemRequests(requestId);
    assertEq(owner, user1, "request remains pending");
    assertEq(usdmAmount, redeemAmount, "request amount remains pending");
```

```
    assertEq(usdm.balanceOf(address(vault)), redeemAmount, "user USDM remains locked");
    assertEq(redeemEscrow.totalOwed(), redeemAmount, "old escrow obligation is stranded");
    assertEq(usdc.balanceOf(address(redeemEscrow)), redeemAmount, "old escrow liquidity is stranded");
    assertEq(usdc.balanceOf(user1), 1_000_000e6 - redeemAmount, "user did not receive redeemed USDC");
}
```

The expected result is that `claimRedeem()` reverts after the escrow switch, even though the old escrow has both the recorded obligation and enough USDC liquidity to pay the user.
Test command:

```bash
forge test --match-test test_submissionValidity -vvv
```

Output:

```bash
Ran 1 test for test/c4/C4Submission.t.sol:C4Submission
[PASS] test_submissionValidity() (gas: 1364198)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 12.12ms (1.93ms CPU time)

Ran 1 test suite in 90.37ms (12.12ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

Recommended Mitigation

Store the escrow address inside each redemption request and use that escrow during claim:

DIFF

```
struct RedeemRequest {
    address owner;
    uint64  cooldownEnd;
    uint256 usdmAmount;
+   address escrow;
}
```

DIFF

```
redeemRequests[requestId] = RedeemRequest({
    owner: msg.sender,
    cooldownEnd: SafeCast.toUint64(block.timestamp + config.redeemCooldown()),
-   usdmAmount: usdmAmount
+   usdmAmount: usdmAmount,
+   escrow: redeemEscrow
});
```

DIFF

```
- IRedeemEscrow(redeemEscrow).payOut(msg.sender, amount);
+ IRedeemEscrow(req.escrow).payOut(msg.sender, amount);
```

L-01 · LOW

Settled yield can become stranded after `yieldEscrow` is changed

SEVERITY	Low
ROOT CAUSE	Settled yield is not bound to the escrow that received it
CONTRACT	MonetrixVault.sol
FUNCTIONS	settle(uint256), distributeYield(), setYieldEscrow(address)
LINES	L364-L383, L508-L511

Vulnerability Details

The yield pipeline in `MonetrixVault` is split into two phases:

1. The Operator calls `settle(uint256 proposedYield)`, and the Vault transfers USDC yield into the current `yieldEscrow`.
2. Later, the Operator calls `distributeYield()`, and the Vault pulls all USDC from the current `yieldEscrow` before distributing it to sUSDM stakers, the Insurance Fund, and the Foundation.

When yield is settled, the USDC is transferred to the current `yieldEscrow` :

SOLIDITY MonetrixVault.sol

```solidity
function settle(uint256 proposedYield) external onlyOperator requireWired nonReentrant whenNotPaused whenOperatorNotPaused {
    require(proposedYield > 0, "zero yield");

    uint256 vaultBal = usdc.balanceOf(address(this));
    uint256 shortfall_ = IRedeemEscrow(redeemEscrow).shortfall();
    uint256 available = vaultBal > shortfall_ ? vaultBal - shortfall_ : 0;
    require(available >= proposedYield, "insufficient EVM USDC");

    IMonetrixAccountant(accountant).settleDailyPnL(proposedYield);
@>  usdc.safeTransfer(yieldEscrow, proposedYield);
    emit YieldCollected(proposedYield);
}
```

However, when yield is distributed, the Vault again uses the **current** `yieldEscrow` pointer:

SOLIDITY MonetrixVault.sol

```solidity
function distributeYield() external nonReentrant onlyOperator requireWired whenNotPaused whenOperatorNotPaused {
@>  uint256 totalYield = IYieldEscrow(yieldEscrow).balance();
    require(totalYield > 0, "no yield");

    uint256 balBefore = usdc.balanceOf(address(this));
@>  IYieldEscrow(yieldEscrow).pullForDistribution(totalYield);
    require(usdc.balanceOf(address(this)) >= balBefore + totalYield, "pull");
    ...
}
```

The `yieldEscrow` address is mutable by the Governor:

SOLIDITY MonetrixVault.sol

```solidity
function setYieldEscrow(address _escrow) external onlyGovernor {
    require(_escrow != address(0), "zero address");
@>  yieldEscrow = _escrow;
    emit YieldEscrowUpdated(_escrow);
}
```

As a result, if governance changes `yieldEscrow` after `settle()` but before `distributeYield()`, the already-settled USDC remains in the old escrow while the Vault starts reading and pulling from the new escrow.
This can leave the settled yield stranded in the old escrow until governance manually points the Vault back to the old escrow or performs a custom migration.

Scenario

1. The protocol has generated real yield.
2. Operator calls `MonetrixVault.settle(50e6)`.
3. The Vault validates the proposed yield through `MonetrixAccountant.settleDailyPnL()`.
4. The Vault transfers `50 USDC` into `oldYieldEscrow`.
5. Before distribution, Governor calls `setYieldEscrow(newYieldEscrow)`.
6. Operator calls `distributeYield()`.
7. The Vault reads `newYieldEscrow.balance()`.
8. Since the settled funds are still in `oldYieldEscrow`, the new escrow balance is zero.
9. `distributeYield()` reverts with `"no yield"`.
10. The settled yield remains stranded in `oldYieldEscrow`, and users do not receive the expected distribution until governance manually fixes the configuration.

This is a configuration/order-of-operations risk. The funds are not stolen, but the protocol loses access to the ordinary distribution path for the yield that was settled into the previous escrow.

Likelihood

Low. The issue requires a privileged `GOVERNOR` action and an inconsistent operational sequence: changing `yieldEscrow` after yield has been settled but before it is distributed. This is not a scenario that an arbitrary attacker can trigger or repeatedly exploit.
Likelihood is Low because the bug depends on privileged roles making an uncommon configuration change at a specific point in the yield lifecycle.

Impact

Low. Users are affected only indirectly: their pending yield distribution can be delayed or skipped until governance restores the old escrow or performs a migration. The issue does not directly steal user principal, does not make user balances insolvent by itself, and is recoverable through privileged intervention.
Impact is Low because the consequences are primarily operational: settled yield becomes temporarily inaccessible through the normal `distributeYield()` flow.

LIKELIHOOD	IMPACT	OVERALL
Low	**Low**	**Low**
Dependent on the actions of the privileged role.	Rewards are not available for a limited time only.	Low × Low.

Proof of Concept

The PoC was added to the official Code4rena template:

SOLIDITY

```
test/c4/C4Submission.t.sol
```

The test performs the following steps:

SOLIDITY Proof of Code

```solidity
function test_submissionValidity() public {
    uint256 principal = 100_000e6;
    uint256 yieldAmount = 50e6;

    // Step 1: create USDM supply so the Accountant annualized-yield cap
    // allows a non-zero settlement.
    _deposit(user1, principal);

    // Step 2: model real strategy profit arriving as extra EVM USDC in the
    // Vault. This creates distributable surplus without minting new USDM.
    usdc.mint(address(vault), yieldAmount);

    // The Accountant settlement gate requires the minimum interval since
    // initializeSettlement().
    vm.warp(block.timestamp + config.redeemCooldown());
```

```

    // Step 3: operator settles yield. The Vault sends USDC to the current
    // YieldEscrow, which is the original yieldEscrow from setUp().
    vm.prank(operator);
    vault.settle(yieldAmount);

    assertEq(usdc.balanceOf(address(yieldEscrow)), yieldAmount, "old yield escrow holds settled yield");

    // Step 4: governance switches the vault to a fresh YieldEscrow before
    // distribution. The settled funds remain in the old escrow.
    YieldEscrow newYieldEscrow = YieldEscrow(
        address(
            new ERC1967Proxy(
                address(new YieldEscrow()),
                abi.encodeCall(YieldEscrow.initialize, (address(usdc), address(vault), address(acl)))
            )
        )
    );

    vm.prank(admin);
    vault.setYieldEscrow(address(newYieldEscrow));

    assertEq(address(vault.yieldEscrow()), address(newYieldEscrow), "vault now points to new yield escrow");
    assertEq(usdc.balanceOf(address(newYieldEscrow)), 0, "new yield escrow has no settled yield");

    // Step 5: distribution now reads the new escrow balance and reverts
    // with "no yield", while the actual settled yield is stranded in the
    // old escrow that the Vault no longer references.
    vm.prank(operator);
    vm.expectRevert(bytes("no yield"));
    vault.distributeYield();

    assertEq(usdc.balanceOf(address(yieldEscrow)), yieldAmount, "settled yield is stranded in old escrow");
    assertEq(usdc.balanceOf(address(newYieldEscrow)), 0, "new escrow still has no yield");
}
```

The expected result is that `distributeYield()` reverts after the escrow switch, even though the previous escrow holds already-settled yield.

Test command:

```bash
forge test --match-test test_submissionValidity -vvv
```

Output:

```bash
Ran 1 test for test/c4/C4Submission.t.sol:C4Submission
[PASS] test_submissionValidity() (gas: 1158617)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 11.01ms (1.52ms CPU time)

Ran 1 test suite in 24.29ms (11.01ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

Recommended Mitigation

Prevent changing `yieldEscrow` while the current escrow still holds undistributed yield:

```diff
function setYieldEscrow(address _escrow) external onlyGovernor {
    require(_escrow != address(0), "zero address");
+   require(IYieldEscrow(yieldEscrow).balance() == 0, "undistributed yield");
    yieldEscrow = _escrow;
    emit YieldEscrowUpdated(_escrow);
}
```

Alternatively, add an explicit migration flow that pulls or transfers undistributed yield from the old escrow before updating the Vault pointer.

L-02 · LOW

Dust first staker can bypass the empty-sUSDM yield reroute and capture most user-side yield

SEVERITY	Low
ROOT CAUSE	Empty-vault yield protection checks only `totalSupply() == 0`, not economically meaningful stake
CONTRACT	MonetrixVault.sol
FUNCTIONS	distributeYield()
LINES	L377-L399

Vulnerability Details

`MonetrixVault.distributeYield()` contains an explicit guard for the case where yield exists but there are no sUSDM stakers:

SOLIDITY MonetrixVault.sol

```solidity
377 function distributeYield() external nonReentrant onlyOperator requireWired
    whenNotPaused whenOperatorNotPaused {
378     uint256 totalYield = IYieldEscrow(yieldEscrow).balance();
379     require(totalYield > 0, "no yield");
380
381     uint256 balBefore = usdc.balanceOf(address(this));
382     IYieldEscrow(yieldEscrow).pullForDistribution(totalYield);
383     require(usdc.balanceOf(address(this)) >= balBefore + totalYield, "pull");
384
385     uint256 userShare = (totalYield * config.userYieldBps()) / 10000;
386     uint256 insuranceShare = (totalYield * config.insuranceYieldBps()) / 10000;
387
388     // Empty-vault yield would be captured by next depositor (L1-H1); reroute to
    foundation.
```

```
@>  if (userShare > 0 && susdm.totalSupply() == 0) {
        userShare = 0;
    }

    uint256 foundationShare = totalYield - userShare - insuranceShare;

    if (userShare > 0) {
        usdm.mint(address(this), userShare);
        IERC20(address(usdm)).forceApprove(address(susdm), userShare);
@>      susdm.injectYield(userShare);
    }
    ...
}
```

The intended behavior is clear from the comment: if sUSDM is empty, user-side yield should not be captured by the next depositor. Instead, that share is rerouted to the Foundation through `foundationShare`.
However, the check only handles the exact zero-supply case:

SOLIDITY MonetrixVault.sol

```
susdm.totalSupply() == 0
```

An attacker can become the first staker with a dust-sized USDM deposit immediately before `distributeYield()` is called. This changes `sUSDM.totalSupply()` from zero to non-zero, causing the empty-vault guard to be bypassed.
Since the attacker owns all sUSDM shares at that point, the injected `userShare` accrues almost entirely to the attacker.
The sUSDM vault uses the USDM balance of the contract as its `totalAssets()`:

SOLIDITY sUSDM.sol

```
function totalAssets() public view override returns (uint256) {
    return IERC20(asset()).balanceOf(address(this));
}
```

and accepts public deposits:

SOLIDITY sUSDM.sol

```solidity
function deposit(uint256 assets, address receiver) public override nonReentrant whenNotPaused returns (uint256) {
    return super.deposit(assets, receiver);
}
```

Therefore, a dust first stake is sufficient to make the Vault treat the sUSDM pool as non-empty for the purpose of yield distribution.

Scenario

1. The protocol has yield waiting in `YieldEscrow`.
2. `sUSDM.totalSupply()` is currently zero, meaning there are no stakers.
3. Under the intended empty-vault logic, if `distributeYield()` were called now, `userShare` would be set to zero and rerouted to the Foundation.
4. Before the Operator calls `distributeYield()`, Alice deposits a tiny amount of USDM into sUSDM, for example `99` base units.
5. `sUSDM.totalSupply()` becomes non-zero.
6. Operator calls `MonetrixVault.distributeYield()`.
7. The empty-vault guard no longer triggers.
8. The Vault mints `userShare` USDM and injects it into sUSDM.
9. Alice owns all sUSDM shares and captures almost all of the injected user-side yield.
10. Alice exits through the normal async unstake flow: - `sUSDM.cooldownShares(...)` - wait `unstakeCooldown` - `sUSDM.claimUnstake(...)`

This does not create insolvency and does not steal principal. However, it lets a dust first staker capture yield that the protocol explicitly intended to reroute away from an empty sUSDM vault.

Likelihood

Low. The scenario depends on narrow conditions:

- yield must already be waiting for distribution,
- sUSDM supply must be zero or economically negligible,
- the Operator must call `distributeYield()` while this condition exists,
- the attacker must enter before the distribution transaction.

This is most realistic around protocol launch, after all stakers have exited, or during an unusual operational sequence where yield remains undistributed while sUSDM is empty.

Likelihood is Low because the condition is not expected to occur frequently during normal protocol operation.

Impact

Low. The attacker can capture most of a single user-side yield distribution with a dust stake when sUSDM is empty. The impact is limited to yield allocation fairness; no user principal is directly stolen, and the attacker still has to exit through the protocol's cooldown flow.

Impact is Low because the issue affects an edge-case distribution state and does not compromise the core USDM backing or redemption system.

LIKELIHOOD	IMPACT	OVERALL
Low	**Low**	**Low**
The condition is not expected to occur frequently.	The issue does not compromise the core of USDM.	Low × Low.

Proof of Concept

The PoC was added to the official Code4rena template:

SOLIDITY

```solidity
test/c4/C4Submission.t.sol
```

The test performs the following steps:

SOLIDITY

```solidity
function test_submissionValidity() public {
    uint256 principal = 100e6;
    uint256 dustStake = 99;
    uint256 totalYield = 50e6;
    uint256 expectedUserShare = (totalYield * config.userYieldBps()) / 10_000;

    // Step 1: user obtains USDM. Only a tiny fraction will be staked.
    _deposit(user1, principal);

    // Step 2: yield is already waiting in YieldEscrow while sUSDM has no
    // stakers. If distributeYield() were called now, userShare would be
    // rerouted to the foundation by the empty-vault guard.
    usdc.mint(address(yieldEscrow), totalYield);
    assertEq(susdm.totalSupply(), 0, "sUSDM starts empty");

    // Step 3: attacker/user becomes the first staker with an economically
    // negligible amount. This flips totalSupply from zero to non-zero.
    vm.startPrank(user1);
    usdm.approve(address(susdm), dustStake);
    susdm.deposit(dustStake, user1);
    vm.stopPrank();

    assertGt(susdm.totalSupply(), 0, "dust stake bypasses the zero-supply guard");

    // Step 4: operator distributes yield. Because totalSupply is non-zero,
    // the staker userShare is injected into sUSDM instead of being rerouted.
    vm.prank(operator);
    vault.distributeYield();

    assertEq(susdm.totalYieldInjected(), expectedUserShare, "userShare was injected into sUSDM");

    // Step 5: the dust first staker owns all shares and can begin unstaking
    // almost all of the injected userShare after the distribution.
    uint256 shares = susdm.balanceOf(user1);
    uint256 claimableAssets = susdm.convertToAssets(shares);

    assertGt(claimableAssets, expectedUserShare * 98 / 100, "dust first staker captures almost all userShare");
    assertGt(claimableAssets, dustStake, "claimable assets include more than the original dust stake");
```

```

    vm.prank(user1);
    uint256 requestId = susdm.cooldownShares(shares);

    vm.warp(block.timestamp + config.unstakeCooldown() + 1);

    uint256 usdmBeforeClaim = usdm.balanceOf(user1);
    vm.prank(user1);
    susdm.claimUnstake(requestId);

    assertEq(usdm.balanceOf(user1), usdmBeforeClaim + claimableAssets, "captured yield is paid out as USDM");
}
```

The expected result is that a dust first staker captures almost all of the `userShare` that would otherwise have been rerouted away from an empty sUSDM vault.
Test command:

```bash
forge test --match-test test_submissionValidity -vvv
```

Output:

```bash
Ran 1 test for test/c4/C4Submission.t.sol:C4Submission
[PASS] test_submissionValidity() (gas: 731307)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.88ms (493.87µs CPU time)

Ran 1 test suite in 16.65ms (2.88ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

Recommended Mitigation

Use an economically meaningful threshold instead of checking only for exact zero supply:

```diff
- if (userShare > 0 && susdm.totalSupply() == 0) {
+ if (userShare > 0 && susdm.totalAssets() < MIN_SUSDM_ASSETS_FOR_YIELD) {
      userShare = 0;
  }
```

Alternatively, use a time-weighted or epoch-based reward mechanism so that users must be staked before the yield accrual/distribution epoch begins in order to receive the corresponding user-side yield.
Another simple mitigation is to require the Operator to skip `distributeYield()` when sUSDM is empty or economically negligible, and only distribute after a minimum meaningful staking base exists.